

```
In [1]: import pandas as pd
from matplotlib import pyplot as plt
import numpy as np
import seaborn as sns
import glob
import os
import re
```

Demographic Metric

Race data

```
In [2]: directory_path = r"C:\Users\jabba\Desktop\Code\machine_learning\AUC_masterca

# Specify the file pattern
file_pattern = os.path.join(directory_path, 'acs_race_*.csv')

# Use glob to find all files matching the pattern
file_list = glob.glob(file_pattern)

# Initialize an empty list to hold the DataFrames
dataframes = []

# Loop through each file, read it into a DataFrame, and append it to the list
for file in file_list:

    # Read the CSV file
    df = pd.read_csv(file)

    # Set Label Grouping as index
    df.set_index('Label (Grouping)', drop=True, inplace=True)

    # Delete Percent columns
    df = df.loc[:, ~df.columns.str.contains('Percent', case=False)]

    # Transpose
    df = df.T

    # Add a year column based on the file name
    year = os.path.basename(file).split('_')[-1].split('.')[0]
    df['Estimate Year'] = year

    # Append the modified DataFrame to the list
    dataframes.append(df)

# Concatenate all DataFrames into a single DataFrame
merged_race_df = pd.concat(dataframes, ignore_index=False)

# Move 'Estimate Year' to the front
column_order = ['Estimate Year'] + [col for col in merged_race_df.columns if
merged_race_df = merged_race_df[column_order]
```

```

# Strip whitespace from column names
merged_race_df.columns = merged_race_df.columns.str.strip()

# Specify columns to keep
columns_to_keep = [
    'Estimate Year',
    'Total:',
    'White alone',
    'Black or African American alone',
    # 'American Indian and Alaska Native alone',
    'Asian alone',
    # 'Native Hawaiian and Other Pacific Islander alone'
]

# Check which columns to keep exist
existing_columns_to_keep = [col for col in columns_to_keep if col in merged_

# Create a filtered DataFrame with only the specified columns
filtered_race = merged_race_df[existing_columns_to_keep]

# Extract the tract number using regex
filtered_race.index = filtered_race.index.str.extract(r'(\d+(\.\d+)?)')[0]

# # Rename index to Tract ID
filtered_race.index.name = 'Tract ID'

# # Optionally save the filtered DataFrame to a new CSV file
filtered_race.to_csv('filtered_race.csv', index=False)

# Reset index to turn the index into a column
filtered_race.reset_index(drop=False, inplace=True)

# Display the filtered DataFrame
filtered_race

```

Out[2]:

	Label (Grouping)	Tract ID	Estimate Year	Total:	White alone	Black or African American alone	Asian alone
	0	4052	2017	5,125	1,930	711	1,519
	1	4053.01	2017	3,019	1,552	757	275
	2	4053.02	2017	2,446	885	367	848
	3	4054.01	2017	4,014	1,061	806	1,362
	4	4054.02	2017	3,250	404	841	1,117
	...	...	...	...	...	...	...
	67	4057	2022	3,522	609	985	934
	68	4058	2022	4,182	630	850	1,815
	69	4059.01	2022	3,757	434	643	1,261
	70	4059.02	2022	3,138	407	234	1,275
	71	4060	2022	4,735	967	910	1,939

72 rows × 6 columns

Age and Sex data

```
In [3]: # Directory path for age and sex data files
directory_path = r"C:\Users\jabba\Desktop\Code\machine_learning\AUC_masterca

# Specify the file pattern
file_pattern = os.path.join(directory_path, 'acs_age_sex_*.csv')

# Use glob to find all files matching the pattern
file_list = glob.glob(file_pattern)

# Initialize an empty list to hold the DataFrames
dataframes = []

# Loop through each file, read it into a DataFrame, and append it to the list
for file in file_list:

    # Read the CSV file
    df = pd.read_csv(file)

    # Set Label Grouping as index
    df.set_index('Label (Grouping)', drop=True, inplace=True)

    # Delete Percent columns
    df = df.loc[:, ~df.columns.str.contains('Percent', case=False)]

    # Rename the columns
    # df.rename(columns=new_column_names, inplace=True)

    # Drop columns that contain 'Male' or 'Female' in their title
```

```

df = df.loc[:, ~df.columns.str.contains('Male|Female', case=False)]

# Transpose
df = df.T

# Add a year column based on the file name
year = os.path.basename(file).split('_')[-1].split('.')[0]
df['Estimate Year'] = year

# Append the modified DataFrame to the list
dataframes.append(df)

# Concatenate all DataFrames into a single DataFrame
merged_age_sex_df = pd.concat(dataframes, ignore_index=False)

# Move 'Estimate Year' to the front
column_order = ['Estimate Year'] + [col for col in merged_age_sex_df.columns
merged_age_sex_df = merged_age_sex_df[column_order]

# Strip whitespace from column names
merged_age_sex_df.columns = merged_age_sex_df.columns.str.strip()

# Specify columns to keep
columns_to_keep = [
    'Estimate Year',
    'Total population',
    'Under 18 years',
    'Median age (years)',
    '18 years and over'
]

# Check which columns to keep exist
existing_columns_to_keep = [col for col in columns_to_keep if col in merged_

# Create a filtered DataFrame with only the specified columns
filtered_age_sex = merged_age_sex_df[existing_columns_to_keep]

# Create a "Sex" column based on the original index values before transposing
sex_values = []

# Re-create the index to get sex information before transposing
original_indices = []
for index in filtered_age_sex.index:
    original_indices.append(index)

# # Create the "Sex" column
# for index in original_indices:
#     if "Male" in index:
#         sex_values.append(1)
#     elif "Female" in index:
#         sex_values.append(0)
#     else:
#         sex_values.append(2) # For combined categories

# # Assign the list to the Sex column
# filtered_age_sex['Sex'] = sex_values

```

```

# Extract numeric parts from the index using str.extract
filtered_age_sex.index = filtered_age_sex.index.str.extract(r'(\d+(\.\d+)?)')

# # Rename index to Tract ID
filtered_age_sex.index.name = 'Tract ID'

# Reset index to turn the index into a column
filtered_age_sex.reset_index(drop=False, inplace=True)

# Optionally save the filtered DataFrame to a new CSV file
filtered_age_sex.to_csv('final_age_sex_data.csv', index=False)

# Display the filtered DataFrame
filtered_age_sex

```

Out[3]:

	Label (Grouping)	Tract ID	Estimate Year	Total population	Under 18 years	Median age (years)	18 years and over
	0	4052	2017	5,125	606	35.8	4,519
	1	4053.01	2017	3,019	326	36.2	2,693
	2	4053.02	2017	2,446	252	40.0	2,194
	3	4054.01	2017	4,014	567	34.9	3,447
	4	4054.02	2017	3,250	713	34.9	2,537
	...	...	...	...	...	...	...
	67	4057	2022	3,522	624	35.6	2,898
	68	4058	2022	4,182	983	36.3	3,199
	69	4059.01	2022	3,757	1,070	33.7	2,687
	70	4059.02	2022	3,138	626	35.3	2,512
	71	4060	2022	4,735	814	43.8	3,921

72 rows × 6 columns

Merge to create demographic metric

```

In [4]: merged_demo = pd.merge(filtered_race, filtered_age_sex, on=['Tract ID', 'Est
merged_demo.to_csv('merged_demographic_data.csv', index=False)

merged_demo

```

Out[4]:

Label (Grouping)	Tract ID	Estimate Year	Total:	White alone	Black or African American alone	Asian alone	Total population	Un ye
0	4052	2017	5,125	1,930	711	1,519	5,125	
1	4053.01	2017	3,019	1,552	757	275	3,019	
2	4053.02	2017	2,446	885	367	848	2,446	
3	4054.01	2017	4,014	1,061	806	1,362	4,014	
4	4054.02	2017	3,250	404	841	1,117	3,250	
...	...	...	...	...	...	...	...	
67	4057	2022	3,522	609	985	934	3,522	
68	4058	2022	4,182	630	850	1,815	4,182	
69	4059.01	2022	3,757	434	643	1,261	3,757	1,
70	4059.02	2022	3,138	407	234	1,275	3,138	
71	4060	2022	4,735	967	910	1,939	4,735	

72 rows × 10 columns

In []:

Housing Metric

Housing Occupancy data

In [5]: `directory_path = r"C:\Users\jabba\Desktop\Code\machine_learning\AUC_masterca`

```
# Specify the file pattern
file_pattern = os.path.join(directory_path, 'acs_housing_occupancy_*.csv')

# Use glob to find all files matching the pattern
file_list = glob.glob(file_pattern)

# Initialize an empty list to hold the DataFrames
dataframes = []

# Rows to remove
rows_to_remove = [
    "RACE AND HISPANIC OR LATINO ORIGIN OF HOUSEHOLDER",
    "One race --",
    "Two or more races",
    "Hispanic or Latino origin",
    "White alone, not Hispanic or Latino",
```

```

    "AGE OF HOUSEHOLDER",
    "EDUCATIONAL ATTAINMENT OF HOUSEHOLDER",
    "YEAR HOUSEHOLDER MOVED INTO UNIT",
]

# Loop through each file, read it into a DataFrame, and append it to the list
for file in file_list:
    df = pd.read_csv(file)

    # Set Label Grouping as index
    df.set_index('Label (Grouping)', drop=True, inplace=True)

    # Drop columns that contain 'Occupied Housing' in their title
    df = df.loc[:, ~df.columns.str.contains('Occupied housing units!!Estimate

df = df.T

    # Add a year column based on the file name
    year = os.path.basename(file).split('_')[-1].split('.')[0]
    df['Estimate Year'] = year

    # Append the modified DataFrame to the list
    dataframes.append(df)

# Concatenate all DataFrames into a single DataFrame
merged_occupancy_df = pd.concat(dataframes, ignore_index=False)

# Move 'Year' to the front
column_order = ['Estimate Year'] + [col for col in merged_occupancy_df.columns
merged_occupancy_df = merged_occupancy_df[column_order]

# Strip whitespace from column names
merged_occupancy_df.columns = merged_occupancy_df.columns.str.strip()

# Specify columns to keep
columns_to_keep = [
    'Estimate Year',
    'Demographic Group',
    'Census Tract 4052 Total occupied housing units'
]

# Save the merged DataFrame to a new CSV file
# merged_occupancy_df.to_csv('filtered_merged_occupancy_df.csv', index=False

## filtered_merged_occupancy_df = merged_occupancy_df.dropna(how='all')
## pd.set_option('display.max_rows', None)
## pd.set_option('display.max_colwidth', None) # Show full column width

# filtered_merged_occupancy_df

```

Median Home Price data

```

In [6]: # Define the directory containing the median home price data
directory_path = r"C:\Users\jabba\Desktop\Code\machine_learning\AUC_masterca

```

```

# Specify the file pattern for median home price files
file_pattern = os.path.join(directory_path, 'acs_median_home_price_*.csv')

# Use glob to find all files matching the pattern
file_list = glob.glob(file_pattern)

# Initialize an empty list to hold the DataFrames
dataframes = []

# Define the tracts for renaming
tracts = [
    '4052', '4053.01', '4053.02', '4054.01',
    '4054.02', '4055', '4056', '4057', '4058',
    '4059.01', '4059.02', '4060'
]

# Function to rename tract columns
def rename_tract_column(col_name):
    for tract in tracts:
        if tract in col_name:
            return f"Tract {tract} Estimate"
    return col_name

# Loop through each file, read it into a DataFrame, and append it to the list
for file in file_list:
    df = pd.read_csv(file)

    # Add a year column based on the file name
    year = os.path.basename(file).split('_')[-1].split('.')[0]
    df['Estimate Year'] = year

    # Rename columns, focusing on tract names
    df.columns = [rename_tract_column(col) for col in df.columns]

    # Append the modified DataFrame to the list
    dataframes.append(df)

# Concatenate all DataFrames into a single DataFrame
merged_housing_df = pd.concat(dataframes, ignore_index=True)

# Move 'Estimate Year' to the front
column_order = ['Estimate Year'] + [col for col in merged_housing_df.columns
                                     if col != 'Estimate Year']
merged_housing_df = merged_housing_df[column_order]

# Remove rows where all elements are NaN
filtered_merged_median_price_df = merged_housing_df.dropna(how='all')

# Rename the 'Label (Grouping)' column to 'Median Value (Dollars)'
filtered_merged_median_price_df.rename(columns={'Label (Grouping)': 'Median Value (Dollars)'})

# Save the merged DataFrame to a new CSV file
#filtered_merged_median_price_df.to_csv('filtered_merged_median_price_df.csv')

# Display the first few rows of the filtered DataFrame
filtered_merged_median_price_df

```

Out[6]:

	Estimate Year	Median Value (Dollars)	Tract 4052 Estimate	Tract 4053.01 Estimate	Tract 4053.02 Estimate	Tract 4054.01 Estimate	Tract 4054.02 Estimate	Esti
0	2017	Median value (dollars)	764,100	647,700	428,300	459,500	428,600	50
1	2018	Median value (dollars)	839,200	627,100	456,300	462,000	466,700	66
2	2019	Median value (dollars)	888,900	682,000	585,900	498,800	573,700	68
3	2020	Median value (dollars)	886,100	779,600	491,700	581,300	648,000	70
4	2021	Median value (dollars)	866,500	878,700	653,800	614,100	731,100	71
5	2022	Median value (dollars)	1,060,300	793,300	637,800	652,400	976,000	75

merge to create housing metric

```
In [7]: # Reshape the housing occupancy DataFrame using melt, focusing on relevant columns
occupancy_melted = filtered_merged_occupancy_df.melt(id_vars=['Estimate Year', 'Tract'],
                                                    value_vars=[
                                                        'Total occupancy',
                                                        'owner-occupied',
                                                        'renter-occupied'
                                                    ],
                                                    var_name='Tract',
                                                    value_name='Occupancy Count')

# Clean the 'Tract' column to match the format in the median home prices DataFrame
occupancy_melted['Tract'] = occupancy_melted['Tract'].str.replace(' Total occupancy', '')
occupancy_melted['Tract'] = occupancy_melted['Tract'].str.replace(' owner-occupied', '')
occupancy_melted['Tract'] = occupancy_melted['Tract'].str.replace(' renter-occupied', '')

# Remove any leading or trailing spaces from the Tract names
occupancy_melted['Tract'] = occupancy_melted['Tract'].str.strip()

# Convert the 'Occupancy Count' to numeric, forcing errors to NaN
occupancy_melted['Occupancy Count'] = pd.to_numeric(occupancy_melted['Occupancy Count'], errors='coerce')

# Aggregate the occupancy counts by Estimate Year
occupancy_aggregated = occupancy_melted.groupby('Estimate Year').agg({
    'Occupancy Count': 'mean'
}).reset_index()
```

```

# Merge the aggregated occupancy DataFrame with median home prices based on
merged_data = pd.merge(occupancy_aggregated, filtered_merged_median_price_df,
                        left_on='Estimate Year',
                        right_on='Estimate Year')

# Rename columns for clarity
merged_data.rename(columns={
    'Median Value (Dollars)': 'Median Home Price'
}, inplace=True)

# Optionally, save the merged dataset for further analysis
merged_data.to_csv('merged_housing_market_data.csv', index=False)
merged_data

```

```

-----
NameError                                Traceback (most recent call last)
Cell In[7], line 2
      1 # Reshape the housing occupancy DataFrame using melt, focusing on re
levant columns
----> 2 occupancy_melted = filtered_merged_occupancy_df.melt(id_vars=['Estim
ate Year'],
      3                                                         value_vars=[
      4                                                         col for c
ol in filtered_merged_occupancy_df.columns
      5                                                         if 'Total
occupied housing units' in col or
      6                                                         'owner
-occupied' in col or
      7                                                         'renter
r-occupied' in col
      8                                                         ],
      9                                                         var_name='Tra
ct',
     10                                                         value_name='O
ccupancy Count')
     12 # Clean the 'Tract' column to match the format in the median home pr
ices DataFrame
     13 occupancy_melted['Tract'] = occupancy_melted['Tract'].str.replace('
Total occupied housing units', '', regex=False)

NameError: name 'filtered_merged_occupancy_df' is not defined

```

Economic Stability

Income Data

```

In [ ]: # Define the directory path
directory_path = r"C:\Users\jabba\Desktop\Code\machine_learning\AUC_masterca

# List of years to include in the concatenation
years = [2017, 2018, 2019, 2020, 2021, 2022]

```

```

# Initialize an empty list to hold DataFrames
dataframes = []

# Loop through the years and read the corresponding CSV files
for year in years:
    file_path = os.path.join(directory_path, f'income_stability_{year}.csv')

    # Read the CSV file
    df = pd.read_csv(file_path)

    # If the year is 2022, replace semicolons with commas in the column names
    if year == 2022:
        df.columns = df.columns.str.replace(';', ', ', regex=False)

    # Add a year column based on the file name
    df['Estimate Year'] = year

    # Append the DataFrame to the list
    dataframes.append(df)

# Concatenate all DataFrames
combined_data = pd.concat(dataframes, ignore_index=True)

# Remove specified rows based on the values in a specific column (assuming the column is 'Estimate Year')
rows_to_drop = [
    "HOUSEHOLD INCOME BY RACE AND HISPANIC OR LATINO ORIGIN OF HOUSEHOLDER",
    "HOUSEHOLD INCOME BY AGE OF HOUSEHOLDER", "FAMILIES", "FAMILY INCOME BY",
    "FAMILY INCOME BY NUMBER OF EARNERS", "NONFAMILY HOUSEHOLDS"
]
combined_data = combined_data[~combined_data.iloc[:, 0].isin(rows_to_drop)]

# Move 'Estimate Year' to the front
column_order = ['Estimate Year'] + [col for col in combined_data.columns if col != 'Estimate Year']
combined_data = combined_data[column_order]

# Remove columns that contain 'Percent' in their names
combined_data = combined_data.loc[:, ~combined_data.columns.str.contains("Percent")]

# Save the combined DataFrame to a new CSV file
output_file_path = os.path.join(directory_path, 'income_stability_combined.csv')
combined_data.to_csv(output_file_path, index=False)

# Display the first few rows of the combined DataFrame
combined_data.head(100)

```

Employment

In []:

Inclusive Growth score

```
In [ ]: IG_data = pd.read_csv('Inclusive_Growth_Score_Data.csv')  
IG_data
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```